Java Multithreading

By- Anupam Shukla

Why Multithreading?

- Improves CPU utilization by performing multiple operations simultaneously.
- Efficient for I/O-bound and high-latency operations.
- Ideal for tasks like gaming, web services, data processing.
- Provides better responsiveness in interactive applications.
- Thread-A **thread** is a lightweight unit of execution within a program. Think of it like a mini-program running inside your main program.

Thread vs Process

Thread	Process
Part of a process	Independent program
Shares memory with other threads	Has its own memory
Fast to create	Slower to create
Light-weight	Heavy-weight
Used in same application	Can be separate applications

🤹 Multitasking vs Multithreading

Multitasking	Multithreading
Running multiple programs	Running multiple threads in one program
More memory required	Less memory usage
Slower	Faster
Example: Chrome + VLC running	Chat app loading + receiving messages

Creating Threads in Java

- 1. Extending Thread class:
- class MyThread extends Thread { public void run() { ... } }
- 2. Implementing Runnable interface:
- class MyRunnable implements Runnable { public void run() { ... } }
- 3. Lambda with Runnable (Java 8+): new Thread(() -> { ... }).start();

```java
package com.engineeringdigest.corejava.threadDemo;

public class LambdaExpression {
    public static void main(String[] args) {
        Runnable runnable = new Runnable() {
            @Override
            public void run() {
                System.out.println("Hello");
            }
        };
        Thread t1 = new Thread(runnable);
        t1.start();
    }
}
```

```java
public class LambdaExpression {
    public static void main(String[] args) {
        Runnable runnable = () -> {
            System.out.println("Hello");
        };
        Thread t1 = new Thread(runnable);
        t1.start();
    }
}
```

```java
public class LambdaExpression {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10; i++){
                System.out.println("Hello world");
            }
        });
        t1.start();
    }
}
```

```java
public class World implements Runnable {
    @Override
    public void run() {
        for (; ; ) {
            System.out.println("World");
        }
    }
}
```

```java
public class Test {
    public static void main(String[] args) {

        World world = new World();
        Thread t1 =new Thread(world);
        t1.start();

        for (; ; ) {
            System.out.println(Thread.currentThread().getName());
        }
    }
}
```

Methods	Description
void start()	This method causes this thread to begin execution. The Java Virtual Machine calls the run method of this thread.
void run()	If this thread was constructed using a separate Runnable run object, then that Runnable object's run method is called.
boolean isAlive()	This method tests if this thread is alive.
String getName()	This method returns the thread's name.
int getPriority()	This method returns the priority of the thread.
void join()	Waits for this thread to die.
static void sleep(long millis)	The method causes the currently executing thread to sleep for the specific number of milliseconds.
String toString()	The method returns a string representation of this thread.

Why we prefer Runnable?

```
package com.engineeringdigest.corejava;

no usages
public class A extends B, Thread{
}
```

Not possible

```
public class A extends B implements Runnable{

}
```

Possible

Main.java

```java
/*****************************************************************************

                    Online Java Compiler.
            Code, Compile, Run and Debug java program online.
Write your code in this editor and press "Run" button to execute it.

*****************************************************************************
class Hello extends Thread
{
    @Override
  public void run()
  {
      for( int i=0; i<100; i++)
      {
          System.out.println(Thread.currentThread().getName());
      }
  }
}

public class Main
{
    public static void main(String[] args) {
        Hello h=new Hello();
        h.start();
//      System.out.println("Hello World");
for(int i=0; i<100; i++ )
      {
          System.out.println(Thread.currentThread().getName());
      }
    }
}
```

```
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
main
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
main
main
main
main
main
main
main
main
main
main
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
Thread-0
main
main
main
main
main
main
main
main
main
main
main
main
```

Thread Lifecycle Explained

- **New:** A thread is in this state when it is created but not yet started.
- **Runnable:** After the start method is called, the thread becomes runnable. It's ready to run and is waiting for CPU time.
- **Running:** The thread is in this state when it is executing.
- **Blocked/Waiting:** A thread is in this state when it is waiting for a resource or for another thread to perform an action.
- **Terminated:** A thread is in this state when it has finished executing.

- New → Runnable → Running → Blocked/Waiting → Terminated.
- start(): moves thread to Runnable.
- run(): thread logic.
- join(): waits for another thread to finish.
- sleep(): pauses thread temporarily.

Note- When a java program starts,one thread begins running immediately,which is called main thread.This thread is responsible for executing the main method.

```java

public class Main extends Thread
{    @Override
  public void run()
  {
      System.out.println("Running");
      try {
          Thread.sleep(200);  //sleep
      } catch(InterruptedException e) {
          System.out.println("ERROR");
      }
  }
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Hello World");
        Main m=new Main();
        System.out.println(m.getState());
        m.start();
        System.out.println("After start thread state="+m.getState());
        Thread.sleep(400);  //main thread sleep
        System.out.println("Now thread state="+m.getState());
        m.join();
        System.out.println("State="+m.getState());

    }
}

```

input

```
Hello World
NEW
Running
After start thread state=RUNNABLE
Now thread state=TERMINATED
State=TERMINATED
```

```java

public class Main extends Thread
{
    public static void main(String[] args) {
        System.out.println("Hello World");
        Main m=new Main();
        System.out.println(m.getState());
        m.start();
        System.out.println("After start thread state="+m.getState());

    }
}
```

```
Hello World
NEW
After start thread state=RUNNABLE
```

getState- is used to know the status of the thread

Example: Basic Thread Code

```
class MyThread extends Thread {
    public void run() {
   System.out.println("Running thread...");
  }
}

public class Test {
  public static void main(String[] args) {
      new MyThread().start();
  }
}
```

Daemon Threads

- **Daemon threads are those threads which work in background.**
- **Syntax- ThreadName.setDaemon(true)**

Synchronization in Java

- Prevents race conditions and ensures thread safety.
- Use synchronized methods or blocks.
- Example: synchronized(this) { ... }
- Only one thread can access a synchronized block at a time.

Code: Synchronization Example

```java
public class WithSync {
    int count = 0;

    synchronized void increment() {
        count++; // Now thread-safe
    }

    public static void main(String[] args) throws InterruptedException {
        WithSync obj = new WithSync();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) obj.increment();
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) obj.increment();
        });

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final Count: " + obj.count); // Always 2000
    }
}
```

wait()
notify()
notifyAll()

Used for inter-thread communication.(threads often need to communicate with each other to accomplish a task).

wait(): pauses for amount of time and releases lock when notified.

notify(): wakes up one waiting thread.

notifyAll(): wakes up all waiting threads.

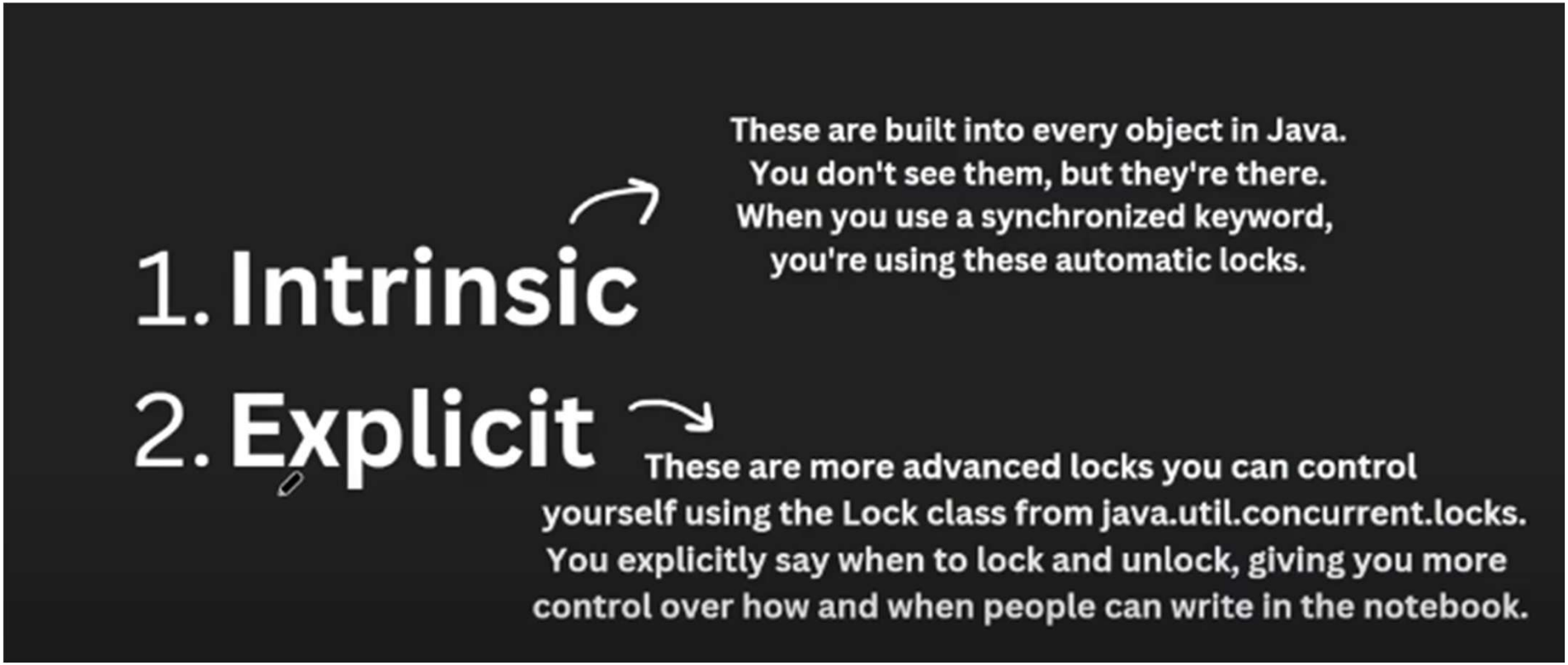
Must be inside synchronized context.

LOCKS

Locks are of two types –

1)Intrinsic (Synchronised lock)

2)Extrinsic Lock(More control when read and write)



Read Write Lock

- ReadWrite lock gives comfortabilibly and allows multiple thread to read resources until anyone is writing to it.

```
public class ReadWriteCounter {
    2 usages
    private int count = 0;
    2 usages
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    2 usages
    private final Lock readLock = lock.readLock();
    2 usages
    private final Lock writeLock = lock.writeLock();
```

ReentrantLock

```
Lock lock = new ReentrantLock();
```

- ReentrantLock from java.util.concurrent.locks.It is an implementation of lock interface.
- Explicit lock/unlock gives better control.
- tryLock() for timeout support.
- Use finally block to ensure unlock().

Synchronised VS ReadWrite Lock VS Reentrant Lock

Feature	Synchronized	ReentrantLock	ReadWriteLock
Type	Intrinsic (built-in)	Explicit lock	Explicit read/write locks
Reentrancy	Yes	Yes	Yes
Fairness Option	No	Yes (new ReentrantLock(true))	Yes (ReentrantReadWriteLock(true))
Interruptible Locking	No	Yes	Yes
Timed Lock	No	Yes (tryLock(timeout))	Yes
Multiple Readers Allowed	No	No	Yes
Use Case	Simple locking	Complex locking needs	High read, low write ratio
Auto Unlock on Exception	Yes	No (needs finally)	No (needs finally)

Volatile Keyword

- Ensures visibility of variable changes across threads.
- Prevents thread-local caching.
- Use for flags, not for atomic operations.
- Example: volatile boolean running = true;

lock.tryLock() in Java

- The tryLock() method is part of the Lock interface in Java's java.util.concurrent.locks package. It attempts to acquire the lock **without blocking**. If the lock is available, it acquires it and returns true; otherwise, it returns false immediately.
- **Key Points:**
  - **Non-blocking:** Unlike lock.lock(), tryLock() does not wait indefinitely for the lock.
  - **Returns a boolean:** It returns true if the lock is acquired, otherwise false.
  - **Can be used with timeout:** The overloaded method tryLock(long time, TimeUnit unit) waits for the specified time before giving up.

```
try {
    if(lock.tryLock( time: 1000, TimeUnit.MILLISECONDS)){
        if(balance >= amount){
            try{
                System.out.println(Thread.currentThread().getName() +
                Thread.sleep( millis: 3000); // Simulate time taken to pr
                balance -= amount;
                System.out.println(Thread.currentThread().getName() +
            }catch (Exception e){

            }finally {
                lock.unlock();
            }
        }else{
```

finally- generally we release resources in finally.

Lock.lock()- same as synchronized- it will wait till we get the lock .So, we **generally don't use it**

Atomic Variables

- Part of java.util.concurrent.atomic package.
- Operations like incrementAndGet() are atomic.
- No need for explicit synchronization.
- Use AtomicInteger, AtomicLong, etc.

What are **Atomic Variables** (aka Atomic Classes)?

Atomic classes in Java are part of the `java.util.concurrent.atomic` package and provide **lock-free, thread-safe operations** on **single variables** using **low-level atomic CPU instructions**.

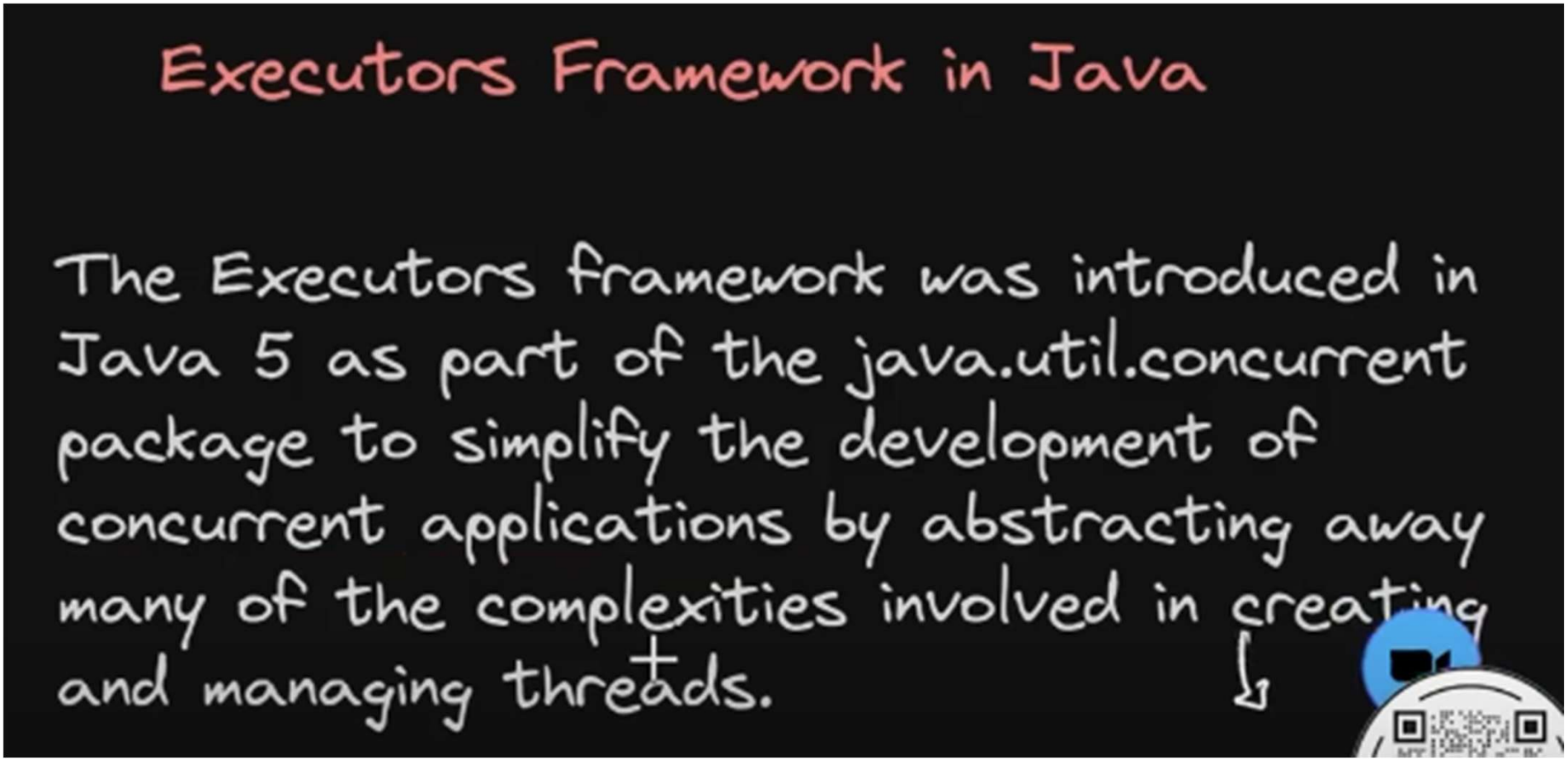
They allow multiple threads to **safely update variables** without using `synchronized`.

AtomicInteger is a thread-safe class in java.util.concurrent.atomic that supports lock-free, atomic operations on an int value — such as incrementing, decrementing, or updating — without using synchronized.

```java
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicExample {
    private static AtomicInteger count = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) {
                count.incrementAndGet();
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final count: " + count.get()); // Should be 2000
    }
}
```

ThreadPools

- Collection of pre-initialized thread that are ready to execute the task.



ExecutorService
& Thread Pool

Better alternative to creating threads manually.

Executors.newFixedThreadPool(n), submit(), shutdown().

Manages thread lifecycle efficiently.

Supports Callable and Future for results.

Create fixed thread pool

```
Executors.newFixedThreadPool( nThreads: 9);
```

Executor is basically a Utility class

```
ExecutorService executorService = Executors.newFixedThreadPool( nThreads: 9);
Executors.newSingleThreadExecutor();
```

newSingleThreadExecutor – in this only one thread executor will come.

Note-Earlier what we write in run we will write in submit .

Executor. Shutdown()-> it stops accepting any submit action and start shutdowning the process.

Factorial program

- If we want each thread to take 3 numbers then in newFixedThreadPool we will write 3 instead of 9.

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {

    public static void main(String[] args) {
        long startTime = System.currentTimeMillis();
        ExecutorService executor = Executors.newFixedThreadPool( nThreads: 9);
        for (int i = 1; i < 10; i++) {
            int finalI = i;
            executor.submit(  () -> {
                long result = factorial(finalI);
                System.out.println(result);
            });
        }
```

Executor

- Executor contains only execute method and didn't had shutdown method.

```
public static void main(String[] args) {
    Executor executor = Executors.newFixedThreadPool( nThreads: 9);
    for (int i = 1; i < 10; i++) {
        int finalI = i;
        executor.execute(() -> {
            long result = factorial(finalI);
            System.out.println(result);
        });
    }
```

Main ×

Callable and Future

- Callable returns a value.
- Future.get() waits for result.
- ExecutorService.submit(Callable) returns Future.
- Useful for parallel computation.

```java
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool( nThreads: 3);
        for (int i = 1; i < 10; i++) {
            int finalI = i;
            Future<?> future = executor.submit(() -> {
                long result = factorial(finalI);
                System.out.println(result);
            });
        }
        executor.shutdown();
    }
```

```java
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) throws ExecutionException, InterruptedException
        ExecutorService executorService = Executors.newSingleThreadExecutor();
        Future<Integer> future = executorService.submit(() -> 42);
        if(future.isDone()){
            System.out.println("Task is done !");
        }
        System.out.println(future.get());
        executorService.shutdown();
    }

}
```

Future give result of submit action

Future methods

- **Future.cancel(true)** means if task is going on or stopped doesn't matter ,cancel it .
- **Future.cancel(false)** means if task is not running then it will not interrupt . if future not running then cancel it.

Method	Description
`get()`	Waits if necessary for the computation to complete and returns the result.
`get(timeout, unit)`	Waits for at most the given time and throws `TimeoutException` if result not available.
`isDone()`	Returns `true` if the task is completed (either normally, cancelled, or failed).
`isCancelled()`	Checks if the task was cancelled.
`cancel(true/false)`	Cancels the task (can interrupt it or not, depending on parameter).

If u want to return something then use callable . And if u don't want to return any thing then use runnable.

- **Executor service-** We studied two types of method –
  - Submit method of runnable
  - Submit method of callable.

```
public static void main(String[] args) throws ExecutionException, InterruptedException {
    ExecutorService executorService = Executors.newSingleThreadExecutor();
    Callable<String> callable = () -> "Hello";
    Future<String> future = executorService.submit(callable);
    System.out.println(future.get());
    executorService.shutdown();
}
```

executorService.invokeAll()=>**collection of tasks** so it uses list **.It can take all tasks and then execute that.**

```java
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) throws ExecutionException, InterruptedException {

        ExecutorService executorService = Executors.newFixedThreadPool( nThreads: 2);

        Callable<Integer> callable1 = () -> 1;
        Callable<Integer> callable2 = () -> 2;
        Callable<Integer> callable3 = () -> 3;

        List<Callable<Integer>> list = Arrays.asList(callable1, callable2, callable3);

        executorService.invokeAll(list);
```

Starvation & Deadlock

- Starvation: thread denied access due to priority.
- Deadlock(blocked forever): two/more threads waiting for each other.It typically occurs when two or more threads have circular dependencies on set of locks.
- Avoid by lock ordering, timeout strategies.

Deadlocks typically occur when four conditions are met simultaneously:

1. Mutual Exclusion: Only one thread can access a resource at a time.
2. Hold and Wait: A thread holding at least one resource is waiting to acquire additional resources held by other threads.
3. No Preemption: Resources cannot be forcibly taken from threads holding them.
4. Circular Wait: A set of threads is waiting for each other in a circular chain.

Producer-Consumer Problem

- One thread produces, another consumes.
- Use BlockingQueue or wait/notify for coordination.
- Prevents overflow/underflow in shared buffer.
- Ensures synchronization and cooperation.

- `newCachedThreadPool()`

A thread pool that **creates new threads as needed** and **reuses idle threads**. Ideal for **many short-lived asynchronous tasks**.

- `CountDownLatch`

A synchronization tool that allows one or more threads to **wait until a set of tasks in other threads completes** by counting down a latch.

- `CyclicBarrier`

A synchronization barrier where a fixed number of threads **wait for each other** to reach a common point before proceeding. **Reusable** after every barrier trip.

- `CompletableFuture`

An enhanced Future that supports **asynchronous, non-blocking programming** with the ability to **chain tasks**, handle exceptions, and **combine multiple futures**.

Summary

- Multithreading improves performance and responsiveness.
- Use synchronization to avoid race conditions.
- ExecutorService simplifies thread management.
- Apply volatile, atomic variables, and locks wisely.
- Mastering multithreading ensures scalable Java applications.
- **Anything is called threadsafe when multiple threads are trying to access but we don’t get unexpected result.**